

Department of Computer Science
University of Pretoria

Programming Languages
COS 333

Practical 4: Functional and Logic Programming

21 April, 2026

1 Objectives

This practical aims to achieve the following general learning objectives:

- To consolidate the concepts covered in Chapter 15 of the prescribed textbook.
- To consolidate the concepts covered in Chapter 16 of the prescribed textbook.

2 Plagiarism Policy

Plagiarism is a serious form of academic misconduct. It involves both appropriating someone else's work and passing it off as one's own work afterwards. Thus, you commit plagiarism when you present someone else's written or creative work (words, images, ideas, opinions, discoveries, artwork, music, recordings, computer-generated work, etc.) as your own. Note that using material produced in whole or part by an AI-based tool (such as ChatGPT) also constitutes plagiarism. Only hand in your own original work. Indicate precisely and accurately when you have used information provided by someone else. Referencing must be done in accordance with a recognised system. Indicate whether you have downloaded information from the Internet. For more details, visit the library's website: <http://www.library.up.ac.za/plagiarism/>.

3 Submission Instructions

Upload your practical-related source code file to the appropriate assignment upload slots on the ClickUP course page. Upload only the source code files for the Scheme and Prolog implementations. Name the Scheme implementation file `u99999999.scm`, and name the Prolog implementation file `u99999999.pl`. In both cases, `99999999` is your student number. Multiple uploads are allowed, but only the last one will be marked. The submission deadline is **Monday, 11 May 2026, at 12:00**.

4 Background Information

You will be implementing your first task in Scheme and your second task in Prolog. Please refer to the background information in practicals 2 and 3 for further information on how to write and interpret Scheme and Prolog programs. DrRacket 8.9 will be used to assess your Scheme submission. SWI-Prolog version 9.0.4 will be used to assess your Prolog submission.

5 Practical Tasks

5.1 Task 1: Scheme

Once again, note the following requirements for this task, which are the same as those listed in the Practical 2 specification:

- You must use the DrRacket interpreter to run and test your submission for this task. For further information on the DrRacket interpreter, please see the specification for Practical 2.
- **Note that you may only use the following built-in Scheme functions and language features in your programs. Racket provides additional functionality that Scheme does not, and you may also not use any of these language features. Failure to observe these rule will result in all marks for a task being forfeited:**

Function construction: `lambda`, `define`

Binding: `let` (more complex `let` functions, liked `let*` and a named `let` are not allowed)

Arithmetic: `+`, `-`, `*`, `/`, `abs`, `sqrt`, `remainder`, `modulo`, `min`, `max`

Boolean values: `#t`, `#f`

Equality predicates: `=`, `>`, `<`, `>=`, `<=`, `even?`, `odd?`, `zero?`, `positive?`, `negative?`, `eqv?`, `eq?`

Logical predicates: `and`, `or`, `not`

List predicates: `list?`, `null?`

Conditionals: `if`, `cond`, `else`

Quoting: `quote`, `'`

Evaluation: `eval`

List manipulation: `list`, `car`, `cdr`, `cons`

Input and output: `display`, `printf`, `newline`, `read`

- You may also define any additional helper functions that you require.

Write a function named `sumAbsNegativeFracPositivesCountZeros`, which receives one parameter that is a simple list containing only numeric values. The `sumAbsNegativeFracPositives` function should yield a sum of converted values contained in the list. If a value, `val`, contained in the list is negative, it should be converted to its absolute value. If `val` is positive, it should be converted to the fraction $1 \div \text{val}$. If `val` is a zero value, it should be counted as 1. For example, the function application

```
(sumAbsNegativeFracPositivesCountZeros '())
```

should yield a value of 0, because the parameter list contains no values that can be added. As another example, the function application

```
(sumAbsNegativeFracPositivesCountZeros '(2 4))
```

should yield a value of 0.75, because all the values in the parameter list are positive, and $1 \div 2 = 0.5$, while $1 \div 4 = 0.25$. Note that the division should produce a fractional value with a decimal point. As a third example, the function application

```
(sumAbsNegativeFracPositivesCountZeros '(-2 -4))
```

should yield a value of 6, because all the values in the parameter list are negative, and the absolute value of -2 is 2, while the absolute value of -4 is 4. As a final example, the function application

```
(sumAbsNegativeFracPositivesCountZeros '(0 2 -4))
```

should yield a value of 5.5 because the zero value is treated as a 1, the positive value 2 is converted to $1 \div 2 = 0.5$, and the negative value -4 is converted to its absolute value of 4.

5.2 Task 2

Once again, take note of the following requirements this task, which are the same as those listed in the Practical 3 specification:

- You must use the SWI-Prolog interpreter to run and test your submission for this task. For further information on the SWI-Prolog interpreter, please see the specification for Practical 3.
- Note that you may only use the simple constants, variables, list manipulation methods, and built-in predicates discussed in the textbook and slides. In particular, do not use if-then, if-then-else, and similar constructs. You may NOT use any more complex predicates provided by the Prolog system itself. In other words, you must write all your own propositions. Failure to observe this rule will result in all marks for a task being forfeited.
- You may implement and use the propositions defined in the textbook and slides (e.g. member, append, and reverse). Note that you must provide the implementation for any of these propositions in your source file. Also note that there are some built-in propositions that correspond to the propositions defined in the textbook, which you may not use.
- You may implement helper propositions if you find them necessary.

Write a Prolog proposition named `absPositivesNegatives` that has two parameters. Both parameters are simple numeric lists (i.e. lists containing only integers). The proposition defines the second parameter to be a list containing the same elements contained in the first parameter, in their original order, with positive and negative values replaced by their absolute values, and zero values removed. Note that the built-in `abs` proposition is not covered in the textbook and slides, and may therefore not be used. To illustrate the use of the `absPositivesNegatives` proposition, consider the following queries and responses:

```
?- absPositivesNegatives([], X).  
X = [].  
  
?- absPositivesNegatives([1, 2, 3], X).  
X = [1, 2, 3].  
  
?- absPositivesNegatives([-1, -2, -3], X).  
X = [1, 2, 3].  
  
?- absPositivesNegatives([-1, 0, 2, 0, -3], X).  
X = [1, 2, 3].
```

Test the `absPositivesNegatives` proposition using the provided example queries, as well as your own tests, and verify that the proposition works as you expect.

6 Marking

Each of the tasks will count 5 marks for a total of 10 marks. Submit the Scheme and Prolog implementations to the appropriate assignment upload slots. Do not upload any additional files other than your source code. Both the implementation and the correct execution of the programs will be taken into account. **You will receive zero for a task that uses a language feature you are not allowed to use.** Your program code will be assessed during the practical session in the week of **Monday, 11 May 2026**.